Verdant Coil – Phase 1.2: Simple Upgrade System

🔍Overview

This phase introduces the foundational system for managing crawler upgrades in Explore Mode. The focus is on toggling upgrade states, integrating with the UI, and preparing hooks for future interaction with environment tiles. It follows an iterative, modular approach that supports clean scaling into later phases.

🕑Goals

- Implement toggleable upgrades using hotkeys (1–3).
- Show upgrade state in the UI via icons.
- Use a centralized UpgradeController for logic.
- Begin hooking upgrade state into crawler behaviour and tile logic.
- Follow all architectural and UI principles outlined in:
- **Verdant Ui Design.pdf**
- **Verdant Coil Design.pdf**
- **Verdant Scene Architecture.pdf**
- **Phase 1 1 Plan.pdf** (as predecessor)

💶Upgrades Introduced

Upgrade	Description	Toggle Key
Hardened Skin	Reduces damage from Acid Pools	1
Acid Sac	Allows digesting Digest Walls	2
Directional Memory	Stub only; will later enable path tracking	3

🔧System Components

1. `UpgradeController.gd`

- Type: Scene-local controller (child of `Crawler.tscn`)
- Role: Tracks and exposes upgrade states
- Exposes methods:

```
enum Upgrade {
    HARDENED_SKIN,
    ACID_SAC,
    DIRECTIONAL_MEMORY,
}
```

```
func has_upgrade(upgrade: Upgrade) -> bool
func toggle_upgrade(upgrade: Upgrade) -> void
signal upgrade_changed(upgrade: Upgrade, value: bool)
```

- Upgrade states are stored as booleans for now, but designed for later conversion to levels.

2. `UpgradeBar.tscn`

- Found under: `ExploreMode.tscn → CanvasLayer`
- Shows one icon per upgrade.
- Icons light up when active.
- Connects to `upgrade_changed` signal for reactivity.
- Icons have tooltips and optional hotkey hints.

3. Input Bindings

- Add to `InputMap`:
- `toggle_upgrade_1` = Key 1
- `toggle_upgrade_2` = Key 2
- `toggle_upgrade_3` = Key 3
- In `_unhandled_input()` (or via UI):

```
if Input.is_action_just_pressed("toggle_upgrade_2"):
    upgrade_controller.toggle_upgrade(UpgradeController.Upgrade.ACID_SAC)
```

4. Crawler Integration

- `Crawler.tscn` has child node `UpgradeController`.
- Movement and tile interaction can begin checking upgrade state:

```
if upgrade_controller.has_upgrade(UpgradeController.Upgrade.ACID_SAC):
    pass_through_digest_wall()
```

- Full tile integration occurs in Phase 1.3.

5. UI Debug Tools (Optional)

- Use `Label` node to show current upgrade states during dev.
- Alternatively, print to console.

⌛Verification Checklist

-

🕐Future-Proofing Notes

- Enum use prevents bugs from string typos.
- Signals enable future visual effects, tile triggers, save/load sync.
- Easily upgraded to global singleton (autoload) if needed in later phases.
- Can be extended to support upgrade levels, energy costs, passive effects, or cooldowns.

🕐Next: Phase 1.3

- Implement placeable environmental tiles
- Use `UpgradeController` to gate access to tile interactions
- Tiles: Digest Wall, Acid Pool, Sticky Web, Heartroot
- TileMap metadata and hazards become fully functional

This concludes the scoped, structured plan for Phase 1.2: enabling a fully testable, modular upgrade system without environmental dependencies.